

PART 2 REACT NATIVE COURSE



Part 2: Core Development (Intermediate Level)

Chapter 4: User Interactions and Events

Handling Touch Events

React Native provides several components for handling user interactions. Let's start with the basic touch events:

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TouchableOpacity,
  TouchableHighlight,
  Pressable,
  Alert,
  StyleSheet,
} from 'react-native';

function TouchEventsExample() {
  const [pressCount, setPressCount] = useState(0);
  const [longPressCount, setLongPressCount] = useState(0);

  // Handle different types of touch events
  const handlePress = () => {
    setPressCount(count => count + 1);
  };

  const handleLongPress = () => {
    setLongPressCount(count => count + 1);
    Alert.alert('Long Press', 'You performed a long press!');
  };

  const handlePressIn = () => {
    console.log('Touch started');
  };

  const handlePressOut = () => {
    console.log('Touch ended');
  };

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Touch Events Example</Text>
```

 Copy

```

    {/* TouchableOpacity - reduces opacity when pressed */}
    <TouchableOpacity
      style={styles.button}
      onPress={handlePress}
      onLongPress={handleLongPress}
      onPressIn={handlePressIn}
      onPressOut={handlePressOut}
      activeOpacity={0.7} // Controls opacity when pressed
    >
      <Text style={styles.buttonText}>
        TouchableOpacity (Pressed: {pressCount})
      </Text>
    </TouchableOpacity>

    {/* TouchableHighlight - shows highlight when pressed */}
    <TouchableHighlight
      style={styles.button}
      onPress={handlePress}
      underlayColor="#3498db" // Color when pressed
    >
      <Text style={styles.buttonText}>TouchableHighlight</Text>
    </TouchableHighlight>

    {/* Pressable - modern, more flexible component */}
    <Pressable
      style={({ pressed }) => [
        styles.button,
        pressed && styles.pressedButton
      ]}
      onPress={handlePress}
      onLongPress={handleLongPress}
    >
      {({ pressed }) => (
        <Text style={styles.buttonText}>
          Pressable {pressed ? '(Pressing...)' : ''}
        </Text>
      )}
    </Pressable>

    <Text style={styles.counter}>
      Long Presses: {longPressCount}
    </Text>
  </View>
);
}

const styles = StyleSheet.create({
  container: {

```

```
    flex: 1,
    padding: 20,
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 30,
  },
  button: {
    backgroundColor: '#2ecc71',
    padding: 15,
    borderRadius: 8,
    marginBottom: 15,
    alignItems: 'center',
  },
  pressedButton: {
    backgroundColor: '#27ae60',
    transform: [{ scale: 0.96 }],
  },
  buttonText: {
    color: 'white',
    fontSize: 16,
    fontWeight: '600',
  },
  counter: {
    fontSize: 18,
    textAlign: 'center',
    marginTop: 20,
  },
});

export default TouchEventsExample;
```

TextInput and Form Handling

Forms are crucial for user input. Here's how to handle text input and forms in React Native:[1]

```
import React, { useState } from 'react';
import {
  View,
  Text,
  TextInput,
  TouchableOpacity,
  Alert,
  ScrollView,
```

 Copy

```
    StyleSheet,
  } from 'react-native';

function FormExample() {
  const [formData, setFormData] = useState({
    name: '',
    email: '',
    phone: '',
    message: '',
  });
  const [errors, setErrors] = useState({});

  // Handle input changes
  const handleInputChange = (field, value) => {
    setFormData(prev => ({
      ...prev,
      [field]: value
    }));

    // Clear error when user starts typing
    if (errors[field]) {
      setErrors(prev => ({
        ...prev,
        [field]: ''
      }));
    }
  };

  // Validate form data
  const validateForm = () => {
    const newErrors = {};

    if (!formData.name.trim()) {
      newErrors.name = 'Name is required';
    }

    if (!formData.email.trim()) {
      newErrors.email = 'Email is required';
    } else if (!isValidEmail(formData.email)) {
      newErrors.email = 'Please enter a valid email';
    }

    if (!formData.phone.trim()) {
      newErrors.phone = 'Phone number is required';
    } else if (formData.phone.length < 10) {
      newErrors.phone = 'Phone number must be at least 10 digits';
    }

    setErrors(newErrors);
  };
}
```

```

    return Object.keys(newErrors).length === 0;
  };

  // Simple email validation
  const isValidEmail = (email) => {
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    return emailRegex.test(email);
  };

  // Handle form submission
  const handleSubmit = () => {
    if (validateForm()) {
      Alert.alert(
        'Form Submitted',
        `Name: ${formData.name}\nEmail: ${formData.email}\nPhone:
${formData.phone}`,
        [{ text: 'OK' }]
      );
      // Reset form
      setFormData({
        name: '',
        email: '',
        phone: '',
        message: '',
      });
    }
  };

  return (
    <ScrollView style={styles.container}>
      <Text style={styles.title}>Contact Form</Text>

      {/* Name Input */}
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Full Name *</Text>
        <TextInput
          style={[styles.input, errors.name && styles.inputError]}
          value={formData.name}
          onChangeText={(text) => handleInputChange('name', text)}
          placeholder="Enter your full name"
          placeholderTextColor="#999"
        />
        {errors.name && <Text style={styles.errorText}>{errors.name}</Text>}
      </View>

      {/* Email Input */}
      <View style={styles.inputContainer}>
        <Text style={styles.label}>Email Address *</Text>
        <TextInput

```

```

    style={styles.input, errors.email && styles.inputError]}
    value={formData.email}
    onChangeText={({text) => handleInputChange('email', text)}}
    placeholder="Enter your email address"
    placeholderTextColor="#999"
    keyboardType="email-address" // Shows email keyboard
    autoCapitalize="none" // Prevents auto capitalization
    autoCorrect={false} // Disables auto correction
  />
  {errors.email && <Text style={styles.errorText}>{errors.email}</Text>}
</View>

{/* Phone Input */}
<View style={styles.inputContainer}>
  <Text style={styles.label}>Phone Number *</Text>
  <TextInput
    style={styles.input, errors.phone && styles.inputError]}
    value={formData.phone}
    onChangeText={({text) => handleInputChange('phone', text)}}
    placeholder="Enter your phone number"
    placeholderTextColor="#999"
    keyboardType="phone-pad" // Shows number keyboard
    maxLength={15} // Limit input length
  />
  {errors.phone && <Text style={styles.errorText}>{errors.phone}</Text>}
</View>

{/* Message Input - Multiline */}
<View style={styles.inputContainer}>
  <Text style={styles.label}>Message</Text>
  <TextInput
    style={styles.input, styles.textArea]}
    value={formData.message}
    onChangeText={({text) => handleInputChange('message', text)}}
    placeholder="Enter your message (optional)"
    placeholderTextColor="#999"
    multiline={true} // Allow multiple lines
    numberOfLines={4} // Initial height
    textAlignVertical="top" // Start text from top
  />
</View>

{/* Submit Button */}
<TouchableOpacity style={styles.submitButton} onPress={handleSubmit}>
  <Text style={styles.submitButtonText}>Submit Form</Text>
</TouchableOpacity>
</ScrollView>
);
}

```

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f8f9fa',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 30,
    color: '#2c3e50',
  },
  inputContainer: {
    marginBottom: 20,
  },
  label: {
    fontSize: 16,
    fontWeight: '600',
    marginBottom: 8,
    color: '#2c3e50',
  },
  input: {
    borderWidth: 1,
    borderColor: '#ddd',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    backgroundColor: 'white',
  },
  inputError: {
    borderColor: '#e74c3c',
  },
  textArea: {
    height: 100,
  },
  errorText: {
    color: '#e74c3c',
    fontSize: 12,
    marginTop: 4,
  },
  submitButton: {
    backgroundColor: '#3498db',
    padding: 15,
    borderRadius: 8,
    alignItems: 'center',
    marginTop: 20,
  },
},
```

```
    submitButtonText: {
      color: 'white',
      fontSize: 18,
      fontWeight: 'bold',
    },
  });

export default FormExample;
```

Chapter 5: Navigation and Screen Management

Navigation is crucial for multi-screen apps. React Navigation is the most popular library for this purpose.[2][3][4]

Installing React Navigation

```
npm install @react-navigation/native
npx expo install react-native-screens react-native-safe-area-context
```

 Copy

Stack Navigator - Basic Screen Transitions

```
// App.js - Main navigation setup
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from './screens/HomeScreen';
import ProfileScreen from './screens/ProfileScreen';
import SettingsScreen from './screens/SettingsScreen';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator
        initialRouteName="Home"
        screenOptions={{
          headerStyle: {
            backgroundColor: '#3498db',
          },
          headerTintColor: 'white',
          headerTitleStyle: {
            fontWeight: 'bold',

```

 Copy

```

    },
  }}
}
>
<Stack.Screen
  name="Home"
  component={HomeScreen}
  options={{ title: 'Home Screen' }}
/>
<Stack.Screen
  name="Profile"
  component={ProfileScreen}
  options={{ title: 'User Profile' }}
/>
<Stack.Screen
  name="Settings"
  component={SettingsScreen}
  options={{
    title: 'Settings',
    headerRight: () => (
      <TouchableOpacity onPress={() => alert('Settings action')}>
        <Text style={{ color: 'white', marginRight: 10 }}>Save</Text>
      </TouchableOpacity>
    ),
  }}
/>
</Stack.Navigator>
</NavigationContainer>
);
}

```

Screen Components with Navigation

```

// screens/HomeScreen.js
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function HomeScreen({ navigation }) {
  // Navigation methods available through props
  const navigateToProfile = () => {
    navigation.navigate('Profile', {
      userId: 123,
      userName: 'John Doe'
    });
  };

  return (
    <View style={styles.container}>

```

 Copy

```

<Text style={styles.title}>Welcome to Home Screen</Text>

<TouchableOpacity
  style={styles.button}
  onPress={navigateToProfile}
>
  <Text style={styles.buttonText}>Go to Profile</Text>
</TouchableOpacity>

<TouchableOpacity
  style={styles.button}
  onPress={() => navigation.navigate('Settings')}
>
  <Text style={styles.buttonText}>Open Settings</Text>
</TouchableOpacity>

<TouchableOpacity
  style={[styles.button, styles.secondaryButton]}
  onPress={() => navigation.push('Home')} // Push another instance
>
  <Text style={styles.buttonText}>Push Another Home</Text>
</TouchableOpacity>
</View>
);
}

// screens/ProfileScreen.js
import React, { useEffect } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';

export default function ProfileScreen({ route, navigation }) {
  // Access passed parameters
  const { userId, userName } = route.params || {};

  // Set navigation options dynamically
  useEffect(() => {
    navigation.setOptions({
      title: userName ? `${userName}'s Profile` : 'Profile',
    });
  }, [navigation, userName]);

  return (
    <View style={styles.container}>
      <Text style={styles.title}>Profile Screen</Text>

      {userId && (
        <Text style={styles.info}>User ID: {userId}</Text>
      )}
    </View>
  );
}

```

```

    {userName && (
      <Text style={styles.info}>User Name: {userName}</Text>
    )}

    <TouchableOpacity
      style={styles.button}
      onPress={() => navigation.goBack()}
    >
      <Text style={styles.buttonText}>Go Back</Text>
    </TouchableOpacity>

    <TouchableOpacity
      style={styles.button}
      onPress={() => navigation.navigate('Settings')}
    >
      <Text style={styles.buttonText}>Go to Settings</Text>
    </TouchableOpacity>

    <TouchableOpacity
      style={[styles.button, styles.dangerButton]}
      onPress={() => navigation.popToTop()} // Go to first screen
    >
      <Text style={styles.buttonText}>Go to Home</Text>
    </TouchableOpacity>
  </View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
    backgroundColor: '#f8f9fa',
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 30,
    color: '#2c3e50',
  },
  info: {
    fontSize: 16,
    marginBottom: 10,
    color: '#34495e',
  },
  button: {
    backgroundColor: '#3498db',

```

```

paddingHorizontal: 20,
paddingVertical: 12,
borderRadius: 8,
marginVertical: 8,
minWidth: 200,
alignItems: 'center',
},
secondaryButton: {
  backgroundColor: '#95a5a6',
},
dangerButton: {
  backgroundColor: '#e74c3c',
},
buttonText: {
  color: 'white',
  fontSize: 16,
  fontWeight: '600',
},
});

```

Tab Navigator - Bottom Navigation

```

// Install additional package
// npm install @react-navigation/bottom-tabs

```

 Copy

```

import React from 'react';
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';

const Tab = createBottomTabNavigator();

export default function TabNavigator() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, color, size }) => {
          let iconName;

          if (route.name === 'Home') {
            iconName = focused ? 'home' : 'home-outline';
          } else if (route.name === 'Profile') {
            iconName = focused ? 'person' : 'person-outline';
          } else if (route.name === 'Settings') {
            iconName = focused ? 'settings' : 'settings-outline';
          }

          return <Ionicons name={iconName} size={size} color={color} />;
        }
      })
    >

```

```

    },
    tabBarActiveTintColor: '#3498db',
    tabBarInactiveTintColor: 'gray',
    headerStyle: {
      backgroundColor: '#3498db',
    },
    headerTintColor: 'white',
  }}}
>
<Tab.Screen name="Home" component={HomeScreen} />
<Tab.Screen name="Profile" component={ProfileScreen} />
<Tab.Screen name="Settings" component={SettingsScreen} />
</Tab.Navigator>
);
}

```

Chapter 6: State Management and Data Flow

Understanding state management is crucial for building scalable React Native applications.[5][6][1]

Local State with useState

```

import React, { useState } from 'react';
import { View, Text, TouchableOpacity, TextInput, StyleSheet } from 'react-native';

function LocalStateExample() {
  // Simple state
  const [count, setCount] = useState(0);

  // Object state
  const [user, setUser] = useState({
    name: '',
    email: '',
    age: 0
  });

  // Array state
  const [todos, setTodos] = useState([]);
  const [newTodo, setNewTodo] = useState('');

  // Updating simple state
  const increment = () => setCount(prev => prev + 1);
  const decrement = () => setCount(prev => prev - 1);

```

 Copy

```

// Updating object state - always create new object
const updateUser = (field, value) => {
  setUser(prevUser => ({
    ...prevUser,
    [field]: value
  }));
};

// Adding to array state
const addTodo = () => {
  if (newTodo.trim()) {
    setTodos(prevTodos => [
      ...prevTodos,
      {
        id: Date.now(),
        text: newTodo.trim(),
        completed: false
      }
    ]);
    setNewTodo('');
  }
};

// Removing from array state
const removeTodo = (id) => {
  setTodos(prevTodos => prevTodos.filter(todo => todo.id !== id));
};

// Toggling todo completion
const toggleTodo = (id) => {
  setTodos(prevTodos =>
    prevTodos.map(todo =>
      todo.id === id
        ? { ...todo, completed: !todo.completed }
        : todo
    )
  );
};

return (
  <View style={styles.container}>
    {/* Counter Example */}
    <View style={styles.section}>
      <Text style={styles.sectionTitle}>Counter: {count}</Text>
      <View style={styles.buttonRow}>
        <TouchableOpacity style={styles.button} onPress={decrement}>
          <Text style={styles.buttonText}>-</Text>
        </TouchableOpacity>
        <TouchableOpacity style={styles.button} onPress={increment}>

```

```

        <Text style={styles.buttonText}>+</Text>
      </TouchableOpacity>
    </View>
  </View>

  { /* User Form Example */ }
  <View style={styles.section}>
    <Text style={styles.sectionTitle}>User Information</Text>
    <TextInput
      style={styles.input}
      placeholder="Name"
      value={user.name}
      onChangeText={({text}) => updateUser('name', text)}
    />
    <TextInput
      style={styles.input}
      placeholder="Email"
      value={user.email}
      onChangeText={({text}) => updateUser('email', text)}
      keyboardType="email-address"
    />
    <TextInput
      style={styles.input}
      placeholder="Age"
      value={user.age.toString()}
      onChangeText={({text}) => updateUser('age', parseInt(text) || 0)}
      keyboardType="numeric"
    />
    <Text style={styles.userInfo}>
      User: {user.name} ({user.email}), Age: {user.age}
    </Text>
  </View>

  { /* Todo List Example */ }
  <View style={styles.section}>
    <Text style={styles.sectionTitle}>Todo List ({todos.length})</Text>
    <View style={styles.todoInput}>
      <TextInput
        style={[styles.input, { flex: 1 }]}
        placeholder="Add new todo"
        value={newTodo}
        onChangeText={setNewTodo}
      />
      <TouchableOpacity style={styles.addButton} onPress={addTodo}>
        <Text style={styles.buttonText}>Add</Text>
      </TouchableOpacity>
    </View>

    {todos.map(todo => (

```

```

    <View key={todo.id} style={styles.todoItem}>
      <TouchableOpacity
        style={styles.todoText}
        onPress={() => toggleTodo(todo.id)}
      >
        <Text style={[
          styles.todoTextContent,
          todo.completed && styles.completedTodo
        ]}>
          {todo.text}
        </Text>
      </TouchableOpacity>
      <TouchableOpacity
        style={styles.deleteButton}
        onPress={() => removeTodo(todo.id)}
      >
        <Text style={styles.buttonText}>x</Text>
      </TouchableOpacity>
    </View>
  )})
</View>
</View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f8f9fa',
  },
  section: {
    marginBottom: 30,
    padding: 15,
    backgroundColor: 'white',
    borderRadius: 8,
  },
  sectionTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    marginBottom: 15,
    color: '#2c3e50',
  },
  buttonRow: {
    flexDirection: 'row',
    justifyContent: 'center',
  },
  button: {
    backgroundColor: '#3498db',

```

```
padding: 15,
margin: 5,
borderRadius: 8,
minWidth: 50,
alignItems: 'center',
},
addButton: {
  backgroundColor: '#2ecc71',
  padding: 10,
  marginLeft: 10,
  borderRadius: 8,
  alignItems: 'center',
},
deleteButton: {
  backgroundColor: '#e74c3c',
  padding: 5,
  borderRadius: 4,
  alignItems: 'center',
  justifyContent: 'center',
  width: 30,
},
buttonText: {
  color: 'white',
  fontWeight: 'bold',
},
input: {
  borderWidth: 1,
  borderColor: '#ddd',
  padding: 10,
  marginBottom: 10,
  borderRadius: 4,
},
userInfo: {
  marginTop: 10,
  padding: 10,
  backgroundColor: '#ecf0f1',
  borderRadius: 4,
},
todoInput: {
  flexDirection: 'row',
  alignItems: 'center',
},
todoItem: {
  flexDirection: 'row',
  alignItems: 'center',
  padding: 10,
  borderBottomWidth: 1,
  borderBottomColor: '#eee',
},
```

```
    todoText: {
      flex: 1,
    },
    todoTextContent: {
      fontSize: 16,
    },
    completedTodo: {
      textDecorationLine: 'line-through',
      color: '#7f8c8d',
    },
  },
});

export default LocalStateExample;
```

Context API for Global State

```
// contexts/AppContext.js
import React, { createContext, useContext, useReducer } from 'react';

// Define initial state
const initialState = {
  user: null,
  theme: 'light',
  settings: {
    notifications: true,
    language: 'en'
  },
  cart: []
};

// Define actions
const AppContext = createContext();

// Reducer function to handle state updates
function appReducer(state, action) {
  switch (action.type) {
    case 'SET_USER':
      return {
        ...state,
        user: action.payload
      };

    case 'SET_THEME':
      return {
        ...state,
        theme: action.payload
      };
  }
}
```

 Copy

```
case 'UPDATE_SETTINGS':
  return {
    ...state,
    settings: {
      ...state.settings,
      ...action.payload
    }
  };

case 'ADD_TO_CART':
  return {
    ...state,
    cart: [...state.cart, action.payload]
  };

case 'REMOVE_FROM_CART':
  return {
    ...state,
    cart: state.cart.filter(item => item.id !== action.payload)
  };

case 'CLEAR_CART':
  return {
    ...state,
    cart: []
  };

default:
  return state;
}
}

// Context Provider Component
export function AppProvider({ children }) {
  const [state, dispatch] = useReducer(appReducer, initialState);

  // Action creators
  const actions = {
    setUser: (user) => dispatch({ type: 'SET_USER', payload: user }),
    setTheme: (theme) => dispatch({ type: 'SET_THEME', payload: theme }),
    updateSettings: (settings) => dispatch({ type: 'UPDATE_SETTINGS', payload:
settings }),
    addToCart: (item) => dispatch({ type: 'ADD_TO_CART', payload: item }),
    removeFromCart: (itemId) => dispatch({ type: 'REMOVE_FROM_CART', payload:
itemId }),
    clearCart: () => dispatch({ type: 'CLEAR_CART' }),
  };
}
```

```
const value = {
  state,
  actions
};

return (
  <AppContext.Provider value={value}>
    {children}
  </AppContext.Provider>
);
}

// Custom hook to use the context
export function useAppContext() {
  const context = useContext(AppContext);
  if (!context) {
    throw new Error('useAppContext must be used within an AppProvider');
  }
  return context;
}
```

Using Context in Components

```
// components/UserProfile.js
import React from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { useAppContext } from '../contexts/AppContext';

export default function UserProfile() {
  const { state, actions } = useAppContext();

  const handleLogin = () => {
    const user = {
      id: 1,
      name: 'John Doe',
      email: 'john@example.com'
    };
    actions.setUser(user);
  };

  const handleLogout = () => {
    actions.setUser(null);
    actions.clearCart();
  };

  const toggleTheme = () => {
    const newTheme = state.theme === 'light' ? 'dark' : 'light';
```

 Copy

```

    actions.setTheme(newTheme);
  };

  return (
    <View style={[
      styles.container,
      { backgroundColor: state.theme === 'dark' ? '#2c3e50' : '#f8f9fa' }
    ]}>
      {state.user ? (
        <View>
          <Text style={[
            styles.welcomeText,
            { color: state.theme === 'dark' ? 'white' : '#2c3e50' }
          ]}>
            Welcome, {state.user.name}!
          </Text>
          <Text style={[
            styles.email,
            { color: state.theme === 'dark' ? '#bdc3c7' : '#7f8c8d' }
          ]}>
            {state.user.email}
          </Text>
          <TouchableOpacity style={styles.button} onPress={handleLogout}>
            <Text style={styles.buttonText}>Logout</Text>
          </TouchableOpacity>
        </View>
      ) : (
        <View>
          <Text style={[
            styles.loginPrompt,
            { color: state.theme === 'dark' ? 'white' : '#2c3e50' }
          ]}>
            Please log in
          </Text>
          <TouchableOpacity style={styles.button} onPress={handleLogin}>
            <Text style={styles.buttonText}>Login</Text>
          </TouchableOpacity>
        </View>
      )}

      <View style={styles.themeSection}>
        <Text style={[
          styles.themeText,
          { color: state.theme === 'dark' ? 'white' : '#2c3e50' }
        ]}>
          Current theme: {state.theme}
        </Text>
        <TouchableOpacity style={styles.button} onPress={toggleTheme}>
          <Text style={styles.buttonText}>Toggle Theme</Text>
        </TouchableOpacity>
      </View>
    </View>
  );
}

```

```

    </TouchableOpacity>
  </View>

  <Text style={[
    styles.cartInfo,
    { color: state.theme === 'dark' ? 'white' : '#2c3e50' }
  ]}>
    Cart items: {state.cart.length}
  </Text>
</View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    justifyContent: 'center',
  },
  welcomeText: {
    fontSize: 24,
    fontWeight: 'bold',
    textAlign: 'center',
    marginBottom: 10,
  },
  email: {
    fontSize: 16,
    textAlign: 'center',
    marginBottom: 20,
  },
  loginPrompt: {
    fontSize: 18,
    textAlign: 'center',
    marginBottom: 20,
  },
  button: {
    backgroundColor: '#3498db',
    padding: 12,
    borderRadius: 8,
    alignItems: 'center',
    marginVertical: 10,
  },
  buttonText: {
    color: 'white',
    fontSize: 16,
    fontWeight: '600',
  },
  themeSection: {
    marginTop: 30,
  },

```

```

padding: 20,
borderTopWidth: 1,
borderTopColor: '#ddd',
},
themeText: {
  fontSize: 16,
  textAlign: 'center',
  marginBottom: 10,
},
cartInfo: {
  fontSize: 16,
  textAlign: 'center',
  marginTop: 20,
},
});

```

Chapter 7: Working with Lists and Data Rendering

Efficiently displaying lists of data is crucial for mobile apps. React Native provides FlatList and SectionList components.^{[7][8][9]}

Basic FlatList Implementation

```

import React, { useState } from 'react';
import {
  View,
  Text,
  FlatList,
  TouchableOpacity,
  Image,
  StyleSheet,
  RefreshControl,
  ActivityIndicator,
} from 'react-native';

// Sample data
const generateSampleData = () => {
  return Array.from({ length: 50 }, (_, index) => ({
    id: index.toString(),
    title: `Item ${index + 1}`,
    description: `This is the description for item ${index + 1}`,
    price: (Math.random() * 100).toFixed(2),
    image: `https://picsum.photos/60/60?random=${index}`,
    category: index % 3 === 0 ? 'Electronics' : index % 3 === 1 ? 'Clothing' :
    'Books',
  }));

```

 Copy

```

    }));
  };

export default function FlatListExample() {
  const [data, setData] = useState(generateSampleData());
  const [refreshing, setRefreshing] = useState(false);
  const [loading, setLoading] = useState(false);

  // Pull to refresh functionality
  const onRefresh = () => {
    setRefreshing(true);
    // Simulate API call
    setTimeout(() => {
      setData(generateSampleData());
      setRefreshing(false);
    }, 1000);
  };

  // Load more data (pagination)
  const loadMoreData = () => {
    if (loading) return;

    setLoading(true);
    // Simulate loading more data
    setTimeout(() => {
      const currentLength = data.length;
      const newData = Array.from({ length: 10 }, (_, index) => ({
        id: (currentLength + index).toString(),
        title: `Item ${currentLength + index + 1}`,
        description: `This is the description for item ${currentLength + index + 1}`,
        price: (Math.random() * 100).toFixed(2),
        image: `https://picsum.photos/60/60?random=${currentLength + index}`,
        category: (currentLength + index) % 3 === 0 ? 'Electronics' :
          (currentLength + index) % 3 === 1 ? 'Clothing' : 'Books',
      }));
      setData(prevData => [...prevData, ...newData]);
      setLoading(false);
    }, 1000);
  };

  // Render each item
  const renderItem = ({ item, index }) => (
    <TouchableOpacity
      style={styles.itemContainer}
      onPress={() => alert(`Selected: ${item.title}`)}
    >
      <Image source={{ uri: item.image }} style={styles.itemImage} />
      <View style={styles.itemContent}>

```

```

    <Text style={styles.itemTitle}>{item.title}</Text>
    <Text style={styles.itemDescription} numberOfLines={2}>
      {item.description}
    </Text>
    <View style={styles.itemFooter}>
      <Text style={styles.itemCategory}>{item.category}</Text>
      <Text style={styles.itemPrice}>${item.price}</Text>
    </View>
  </View>
</TouchableOpacity>
);

// Render separator between items
const renderSeparator = () => (
  <View style={styles.separator} />
);

// Render footer (loading indicator)
const renderFooter = () => {
  if (!loading) return null;
  return (
    <View style={styles.footerLoader}>
      <ActivityIndicator size="large" color="#3498db" />
      <Text style={styles.loadingText}>Loading more...</Text>
    </View>
  );
};

// Render empty list
const renderEmpty = () => (
  <View style={styles.emptyContainer}>
    <Text style={styles.emptyText}>No items found</Text>
  </View>
);

return (
  <View style={styles.container}>
    <Text style={styles.header}>Product List ({data.length} items)</Text>

    <FlatList
      data={data}
      renderItem={renderItem}
      keyExtractor={({item}) => item.id}
      ItemSeparatorComponent={renderSeparator}
      ListFooterComponent={renderFooter}
      ListEmptyComponent={renderEmpty}
      refreshControl={
        <RefreshControl
          refreshing={refreshing}

```

```

        onRefresh={onRefresh}
        colors={['#3498db']} // Android
        tintColor="#3498db"    // iOS
      />
    }
    onEndReached={loadMoreData}
    onEndReachedThreshold={0.1} // Load more when 10% from bottom
    showsVerticalScrollIndicator={false}
    // Performance optimizations
    removeClippedSubviews={true}
    maxToRenderPerBatch={10}
    windowSize={10}
    initialNumToRender={10}
    getItemLayout={(data, index) => ({
      length: 80,
      offset: 80 * index,
      index,
    })}
  />
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f8f9fa',
  },
  header: {
    fontSize: 20,
    fontWeight: 'bold',
    padding: 15,
    backgroundColor: 'white',
    textAlign: 'center',
    color: '#2c3e50',
  },
  itemContainer: {
    flexDirection: 'row',
    padding: 15,
    backgroundColor: 'white',
    alignItems: 'center',
  },
  itemImage: {
    width: 60,
    height: 60,
    borderRadius: 8,
    marginRight: 15,
  },
  itemContent: {

```

```
    flex: 1,
  },
  itemTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#2c3e50',
    marginBottom: 4,
  },
  itemDescription: {
    fontSize: 14,
    color: '#7f8c8d',
    lineHeight: 18,
    marginBottom: 8,
  },
  itemFooter: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  itemCategory: {
    fontSize: 12,
    color: '#3498db',
    backgroundColor: '#ecf0f1',
    paddingHorizontal: 8,
    paddingVertical: 2,
    borderRadius: 10,
  },
  itemPrice: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#e74c3c',
  },
  separator: {
    height: 1,
    backgroundColor: '#ecf0f1',
    marginHorizontal: 15,
  },
  footerLoader: {
    padding: 20,
    alignItems: 'center',
  },
  loadingText: {
    marginTop: 10,
    color: '#7f8c8d',
  },
  emptyContainer: {
    padding: 50,
    alignItems: 'center',
  },
},
```

```
emptyText: {
  fontSize: 16,
  color: '#7f8c8d',
},
});
```

Chapter 8: Networking and API Integration

Making API calls is essential for most mobile apps. React Native provides the Fetch API for networking.[10][11][12]

Basic API Integration with Fetch

```
import React, { useState, useEffect } from 'react';
import {
  View,
  Text,
  FlatList,
  ActivityIndicator,
  Alert,
  TouchableOpacity,
  TextInput,
  StyleSheet,
} from 'react-native';

// API service functions
const API_BASE_URL = 'https://jsonplaceholder.typicode.com';

export default function APIExample() {
  const [posts, setPosts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const [newPost, setNewPost] = useState({ title: '', body: '' });

  // Fetch all posts
  const fetchPosts = async () => {
    try {
      setLoading(true);
      setError(null);

      const response = await fetch(`${API_BASE_URL}/posts`);

      // Check if request was successful
      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }
    }
  }
```

 Copy

```
}

const data = await response.json();
setPosts(data.slice(0, 10)); // Limit to first 10 posts

} catch (err) {
  setError(err.message);
  Alert.alert('Error', `Failed to fetch posts: ${err.message}`);
} finally {
  setLoading(false);
}
};

// Create new post
const createPost = async () => {
  if (!newPost.title.trim() || !newPost.body.trim()) {
    Alert.alert('Validation Error', 'Please fill in both title and body');
    return;
  }

  try {
    setLoading(true);

    const response = await fetch(`${API_BASE_URL}/posts`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        title: newPost.title,
        body: newPost.body,
        userId: 1,
      }),
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const createdPost = await response.json();

    // Add new post to the beginning of the list
    setPosts(prevPosts => [createdPost, ...prevPosts]);

    // Clear form
    setNewPost({ title: '', body: '' });

    Alert.alert('Success', 'Post created successfully!');
```

```
    } catch (err) {
      Alert.alert('Error', `Failed to create post: ${err.message}`);
    } finally {
      setLoading(false);
    }
  };

// Update post
const updatePost = async (postId, updatedData) => {
  try {
    const response = await fetch(`${API_BASE_URL}/posts/${postId}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(updatedData),
    });

    if (!response.ok) {
      throw new Error(`HTTP error! status: ${response.status}`);
    }

    const updatedPost = await response.json();

    // Update the post in local state
    setPosts(prevPosts =>
      prevPosts.map(post =>
        post.id === postId ? { ...post, ...updatedPost } : post
      )
    );

    Alert.alert('Success', 'Post updated successfully!');

  } catch (err) {
    Alert.alert('Error', `Failed to update post: ${err.message}`);
  }
};

// Delete post
const deletePost = async (postId) => {
  Alert.alert(
    'Confirm Delete',
    'Are you sure you want to delete this post?',
    [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Delete',
        style: 'destructive',
        onPress: async () => {

```

```

    try {
      const response = await fetch(`${API_BASE_URL}/posts/${postId}`, {
        method: 'DELETE',
      });

      if (!response.ok) {
        throw new Error(`HTTP error! status: ${response.status}`);
      }

      // Remove post from local state
      setPosts(prevPosts => prevPosts.filter(post => post.id !== postId));

      Alert.alert('Success', 'Post deleted successfully!');

    } catch (err) {
      Alert.alert('Error', `Failed to delete post: ${err.message}`);
    }
  }
}

];
);
};

// Load posts on component mount
useEffect(() => {
  fetchPosts();
}, []);

// Render post item
const renderPost = ({ item }) => (
  <View style={styles.postItem}>
    <Text style={styles.postTitle}>{item.title}</Text>
    <Text style={styles.postBody} numberOfLines={3}>
      {item.body}
    </Text>
    <View style={styles.postActions}>
      <TouchableOpacity
        style={[styles.actionButton, styles.editButton]}
        onPress={() => updatePost(item.id, { title: item.title + ' (Updated)'
      >
        <Text style={styles.actionButtonText}>Edit</Text>
      </TouchableOpacity>
      <TouchableOpacity
        style={[styles.actionButton, styles.deleteButton]}
        onPress={() => deletePost(item.id)}
      >
        <Text style={styles.actionButtonText}>Delete</Text>
      </TouchableOpacity>
    </View>
  </View>
);
}

```

```

    </View>
  </View>
);

if (loading && posts.length === 0) {
  return (
    <View style={styles.centerContainer}>
      <ActivityIndicator size="large" color="#3498db" />
      <Text style={styles.loadingText}>Loading posts...</Text>
    </View>
  );
}

if (error && posts.length === 0) {
  return (
    <View style={styles.centerContainer}>
      <Text style={styles.errorText}>Error: {error}</Text>
      <TouchableOpacity style={styles.retryButton} onPress={fetchPosts}>
        <Text style={styles.retryButtonText}>Retry</Text>
      </TouchableOpacity>
    </View>
  );
}

return (
  <View style={styles.container}>
    { /* Create Post Form */ }
    <View style={styles.createForm}>
      <Text style={styles.formTitle}>Create New Post</Text>
      <TextInput
        style={styles.input}
        placeholder="Post title"
        value={newPost.title}
        onChangeText={(text) => setNewPost(prev => ({ ...prev, title: text })))}
      />
      <TextInput
        style={[styles.input, styles.textArea]}
        placeholder="Post body"
        multiline
        numberOfLines={3}
        value={newPost.body}
        onChangeText={(text) => setNewPost(prev => ({ ...prev, body: text })))}
      />
      <TouchableOpacity
        style={styles.createButton}
        onPress={createPost}
        disabled={loading}
      >
        <Text style={styles.createButtonText}>

```

```

        {loading ? 'Creating...' : 'Create Post'}
      </Text>
    </TouchableOpacity>
  </View>

  {/* Posts List */}
  <FlatList
    data={posts}
    renderItem={renderPost}
    keyExtractor={({item}) => item.id.toString()}
    onRefresh={fetchPosts}
    refreshing={loading}
    contentContainerStyle={styles.listContainer}
    showsVerticalScrollIndicator={false}
  />
</View>
);
}

```

```

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#f8f9fa',
  },
  centerContainer: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
  },
  loadingText: {
    marginTop: 10,
    fontSize: 16,
    color: '#7f8c8d',
  },
  errorText: {
    fontSize: 16,
    color: '#e74c3c',
    textAlign: 'center',
    marginBottom: 20,
  },
  retryButton: {
    backgroundColor: '#3498db',
    paddingHorizontal: 20,
    paddingVertical: 10,
    borderRadius: 8,
  },
  retryButtonText: {
    color: 'white',
  },

```

```
    fontSize: 16,
    fontWeight: '600',
  },
  createForm: {
    backgroundColor: 'white',
    padding: 15,
    marginBottom: 10,
  },
  formTitle: {
    fontSize: 18,
    fontWeight: 'bold',
    marginBottom: 15,
    color: '#2c3e50',
  },
  input: {
    borderWidth: 1,
    borderColor: '#ddd',
    borderRadius: 8,
    padding: 10,
    marginBottom: 10,
    fontSize: 16,
  },
  textArea: {
    height: 80,
    textAlignVertical: 'top',
  },
  createButton: {
    backgroundColor: '#2ecc71',
    padding: 12,
    borderRadius: 8,
    alignItems: 'center',
  },
  createButtonText: {
    color: 'white',
    fontSize: 16,
    fontWeight: '600',
  },
  listContainer: {
    padding: 10,
  },
  postItem: {
    backgroundColor: 'white',
    padding: 15,
    marginBottom: 10,
    borderRadius: 8,
    elevation: 2,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
```

```

    shadowRadius: 4,
  },
  postTitle: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#2c3e50',
    marginBottom: 8,
  },
  postBody: {
    fontSize: 14,
    color: '#7f8c8d',
    lineHeight: 20,
    marginBottom: 12,
  },
  postActions: {
    flexDirection: 'row',
    justifyContent: 'flex-end',
  },
  actionButton: {
    paddingHorizontal: 12,
    paddingVertical: 6,
    borderRadius: 4,
    marginLeft: 8,
  },
  editButton: {
    backgroundColor: '#f39c12',
  },
  deleteButton: {
    backgroundColor: '#e74c3c',
  },
  actionButtonText: {
    color: 'white',
    fontSize: 12,
    fontWeight: '600',
  },
});

```

This completes Part 2 of the React Native course, covering user interactions, navigation, state management, list rendering, and API integration. These are the core intermediate concepts you'll need to build robust React Native applications.^{[3][4][6][8][11][9][12][2][5][7][10][1]}

Key Takeaways from Part 2:

- Touch events and form handling for user interactions
- Navigation patterns with React Navigation library
- State management techniques from local to global scope
- Efficient list rendering with FlatList
- API integration using the Fetch API

Ready for **Part 3: Advanced Features** when you are!

[1](#) [2](#) [3](#) [4](#) [5](#) [6](#) [7](#) [8](#) [9](#) [10](#) [11](#) [12](#) [13](#) [14](#) [15](#) [16](#) [17](#) [18](#) [19](#) [20](#) [21](#) [22](#) [23](#) [24](#) [25](#) [26](#) [27](#) [28](#) [29](#) [30](#) [31](#) [32](#) [33](#) [34](#) [35](#) [36](#) [37](#) [38](#) [39](#) [40](#)